snickr: Relational Database Design and Web Application

CS6083 — Final Project

Shichen Zhang (sz4968@nyu.edu)
Zihang Yu (zy3493@nyu.edu)

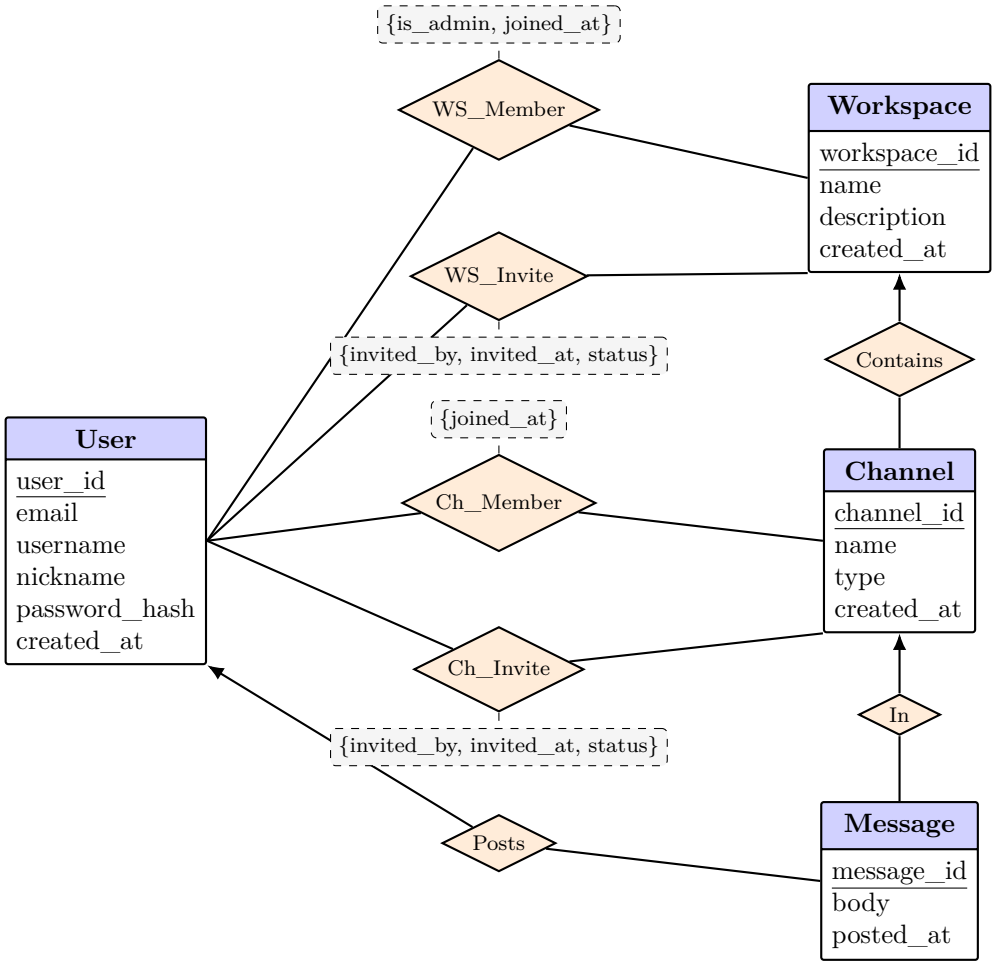
May 15, 2026

Introduction

snickr is a Slack-like collaboration system where users create workspaces, invite members, create public/private/direct channels, and exchange messages inside those channels. This report presents the design and implementation of snickr, covering the relational schema (ER model, DDL, required queries, and sample data) and the Flask-based web application built on top of it.

(a) Database Schema Design

ER diagram.



Relational schema.

- **User**(user_id, email, username, nickname, password_hash, created_at)
  UNIQUE(email), UNIQUE(username).
- **Workspace**(workspace_id, name, description, created_by, created_at)
  FK: created_by → User(user_id); UNIQUE(created_by, name).
- **WorkspaceMember**(workspace_id, user_id, is_admin, joined_at)
  FKs: workspace_id → Workspace, user_id → User.
- **WorkspaceInvitation**(workspace_id, invitee_user_id, invited_by, invited_at, status)
  FKs: workspace_id → Workspace; invitee_user_id, invited_by → User.
- **Channel**(channel_id, workspace_id, name, type, created_by, created_at)
  FKs: workspace_id → Workspace, created_by → User; UNIQUE(workspace_id, name).
- **ChannelMember**(channel_id, user_id, joined_at)
  FKs: channel_id → Channel, user_id → User.
- **ChannelInvitation**(channel_id, invitee_user_id, invited_by, invited_at, status)
  FKs: channel_id → Channel; invitee_user_id, invited_by → User.
- **Message**(message_id, channel_id, sender_user_id, body, posted_at)
  FKs: channel_id → Channel, sender_user_id → User.

Assumptions. Email and username are globally unique; workspace names are unique per creator; channel names are unique only within a workspace; admin is a boolean on workspace membership rather than a separate entity; public channels can be joined by any workspace member, while private channels require an invitation from the channel's creator; a direct channel is a channel of type `direct` with exactly two members (cardinality enforced in the application); channel invitations target only users who are already members of the channel's workspace (enforced in the application); timestamps are stored on every action-bearing row.

Design justification.

- **One Channel table with a `type` enum** rather than separate tables for public, private, and direct channels. All three share the same attributes, so subtyping would add joins without adding structure.
- **Admin is a boolean on WorkspaceMember**, not a separate entity. Every admin is already a member, so a dedicated **Admin** table would duplicate the membership relationship.
- **Separate membership and invitation tables.** Membership records an accepted state with relationship-level attributes (`is_admin`, `joined_at`); invitation records a pending state with its own attributes (`invited_by`, `invited_at`, `status`). Mixing them in one table would conflate "joined" with "invited but not yet joined".
- **Separate invitation tables for workspaces and channels.** They point at different parents, so keeping them apart lets each table declare a clean foreign key.
- **Composite natural keys** `(workspace_id, user_id)` and `(channel_id, user_id)` on the four membership / invitation tables. No surrogate id is needed and the "one row per pair" invariant is structural rather than an extra `UNIQUE` constraint.

Procedures / Application Logic. This design does not rely on database stored procedures or triggers. Multi-step operations are handled by the application using ordinary SQL statements. For example, creating a public channel first inserts the channel and then inserts the creator into `channel_members` if the first step succeeds. Likewise, rules such as "a direct channel has exactly two members" and "channel invitations are issued only to members of the channel's workspace" are checked by the application before the corresponding insert operations are executed.

(b) SQL Schema (DDL)

```
CREATE TABLE users (
    user_id       BIGSERIAL    PRIMARY KEY,
    email         VARCHAR(255) NOT NULL UNIQUE,
    username      VARCHAR(64)  NOT NULL UNIQUE,
    nickname      VARCHAR(64)  NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    created_at    TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE workspaces (
    workspace_id BIGSERIAL    PRIMARY KEY,
    name         VARCHAR(128) NOT NULL,
    description  TEXT,
    created_by   BIGINT       NOT NULL REFERENCES users(user_id),
```

```
    created_at   TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (created_by, name)
);

CREATE TABLE workspace_members (
    workspace_id BIGINT    NOT NULL REFERENCES workspaces(workspace_id),
    user_id      BIGINT    NOT NULL REFERENCES users(user_id),
    is_admin     BOOLEAN   NOT NULL DEFAULT FALSE,
    joined_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (workspace_id, user_id)
);

CREATE TABLE workspace_invitations (
    workspace_id    BIGINT    NOT NULL REFERENCES
       workspaces(workspace_id),
    invitee_user_id BIGINT    NOT NULL REFERENCES users(user_id),
    invited_by      BIGINT    NOT NULL REFERENCES users(user_id),
    invited_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status          VARCHAR(16) NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending','accepted','declined')),
    PRIMARY KEY (workspace_id, invitee_user_id)
);

CREATE TABLE channels (
    channel_id   BIGSERIAL    PRIMARY KEY,
    workspace_id BIGINT       NOT NULL REFERENCES
       workspaces(workspace_id),
    name         VARCHAR(128) NOT NULL,
    type         VARCHAR(16)  NOT NULL
                 CHECK (type IN ('public','private','direct')),
    created_by   BIGINT       NOT NULL REFERENCES users(user_id),
    created_at   TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (workspace_id, name)
);

CREATE TABLE channel_members (
    channel_id BIGINT    NOT NULL REFERENCES channels(channel_id),
    user_id    BIGINT    NOT NULL REFERENCES users(user_id),
    joined_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (channel_id, user_id)
);

CREATE TABLE channel_invitations (
    channel_id      BIGINT    NOT NULL REFERENCES channels(channel_id),
    invitee_user_id BIGINT    NOT NULL REFERENCES users(user_id),
    invited_by      BIGINT    NOT NULL REFERENCES users(user_id),
    invited_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status          VARCHAR(16) NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending','accepted','declined')),
    PRIMARY KEY (channel_id, invitee_user_id)
);

CREATE TABLE messages (
    message_id     BIGSERIAL PRIMARY KEY,
```

```
    channel_id      BIGINT    NOT NULL REFERENCES channels(channel_id),
    sender_user_id  BIGINT    NOT NULL REFERENCES users(user_id),
    body            TEXT      NOT NULL,
    posted_at       TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_messages_channel_time ON messages(channel_id,
   posted_at);
CREATE INDEX idx_messages_sender       ON messages(sender_user_id);
```

(c) Required SQL Queries

(1) Create a new user account

```
INSERT INTO users (email, username, nickname, password_hash)
VALUES (:email, :username, :nickname, :password_hash)
RETURNING user_id;
```

(2) Create a public channel (with authorization check)

```
INSERT INTO channels (workspace_id, name, type, created_by)
SELECT :workspace_id, :name, 'public', :user_id
WHERE EXISTS (
    SELECT 1 FROM workspace_members
    WHERE workspace_id = :workspace_id AND user_id = :user_id
)
RETURNING channel_id;

INSERT INTO channel_members (channel_id, user_id)
VALUES (:new_channel_id, :user_id);
```

Here `:new_channel_id` is the `channel_id` returned by the first insert. The second insert is executed by the application only if the first insert returns a `channel_id`.

(3) For each workspace, list all administrators

```
SELECT w.workspace_id, w.name AS workspace, u.username AS administrator
FROM   workspaces w
JOIN   workspace_members wm ON wm.workspace_id = w.workspace_id
JOIN   users u              ON u.user_id       = wm.user_id
WHERE  wm.is_admin = TRUE
ORDER BY w.workspace_id, u.username;
```

(4) Per public channel: # invitees invited >5 days ago, not yet joined

```
SELECT c.channel_id, c.name,
       COUNT(ci.invitee_user_id) FILTER (WHERE cm.user_id IS NULL)
       AS stale_unjoined_invites
FROM   channels c
LEFT   JOIN channel_invitations ci
```

```
       ON ci.channel_id = c.channel_id
      AND ci.invited_at < CURRENT_TIMESTAMP - INTERVAL '5 days'
LEFT   JOIN channel_members cm
       ON cm.channel_id = ci.channel_id AND cm.user_id =
          ci.invitee_user_id
WHERE  c.workspace_id = :workspace_id
  AND  c.type = 'public'
GROUP BY c.channel_id, c.name
ORDER BY c.name;
```

(5) All messages in a channel, chronological order

```
SELECT m.message_id, m.posted_at, u.username, m.body
FROM   messages m
JOIN   users u ON u.user_id = m.sender_user_id
WHERE  m.channel_id = :channel_id
ORDER BY m.posted_at ASC, m.message_id ASC;
```

(6) All messages posted by a given user

```
SELECT m.message_id, w.name AS workspace, c.name AS channel,
       m.posted_at, m.body
FROM   messages m
JOIN   channels   c ON c.channel_id   = m.channel_id
JOIN   workspaces w ON w.workspace_id = c.workspace_id
WHERE  m.sender_user_id = :user_id
ORDER BY m.posted_at DESC;
```

(7) Accessible messages containing "perpendicular"

```
SELECT m.message_id, w.name AS workspace, c.name AS channel,
       u.username AS author, m.posted_at, m.body
FROM   messages m
JOIN   channels         c  ON c.channel_id    = m.channel_id
JOIN   workspaces       w  ON w.workspace_id  = c.workspace_id
JOIN   users            u  ON u.user_id       = m.sender_user_id
JOIN   channel_members  cm ON cm.channel_id   = c.channel_id
                          AND cm.user_id      = :user_id
JOIN   workspace_members wm ON wm.workspace_id = w.workspace_id
                           AND wm.user_id      = :user_id
WHERE  m.body ILIKE '%perpendicular%'
ORDER BY m.posted_at DESC;
```

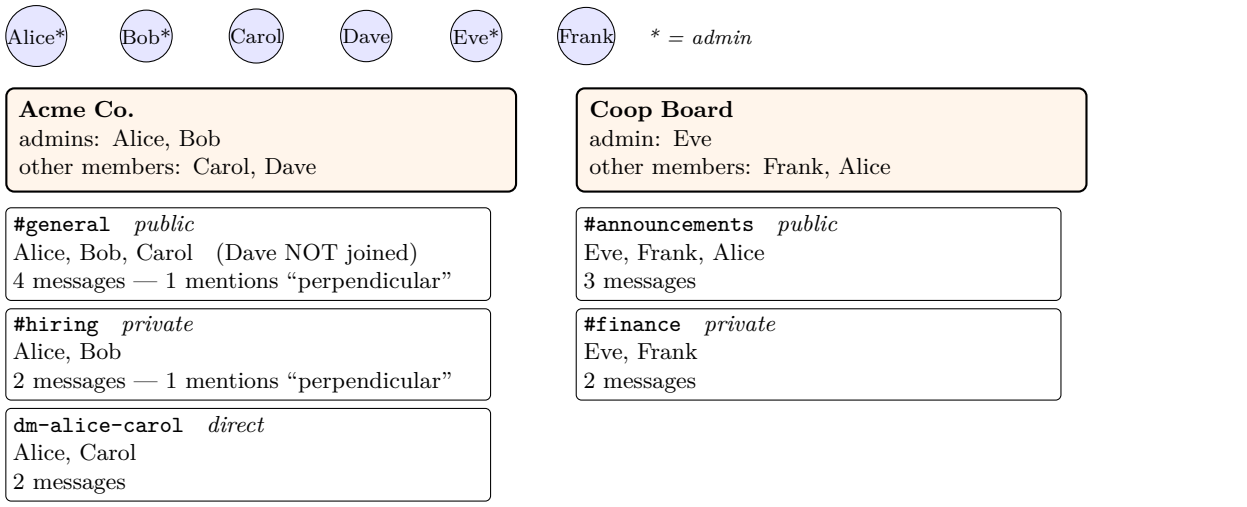
(d) Sample Data and Testing

Sample Data

The sample data populates 6 users, 2 workspaces, 5 channels, 13 messages, and 3 pending invitations. The diagram below summarizes the data as a small workspace map. Alice, Bob, Carol, Dave, Eve,

and Frank are the users; asterisks mark users who are administrators in at least one workspace. The first workspace, Acme Co., has Alice and Bob as admins and Carol and Dave as regular members. It contains a public `#general` channel, a private `#hiring` channel, and a direct channel between Alice and Carol. Dave is listed as an Acme workspace member but is intentionally not a member of any Acme channel.

The second workspace, Coop Board, has Eve as admin and Frank and Alice as regular members, so Alice appears in both workspaces. Coop Board contains a public `#announcements` channel and a private `#finance` channel. The message counts are shown inside each channel box, and channels include notes about whether any message contains the word "perpendicular". The dashed invitation box records three still-pending invitations to Dave: one stale channel invitation in Acme, one recent channel invitation in Acme, and one workspace invitation in Coop Board.



Query Testing

After loading the sample data, the queries in part (c) were executed against the populated database. The observed results are shown below.

(1) Create a new user account

This test inserts a new user named Grace and returns the generated `user_id`.

Query.

```
INSERT INTO users (email, username, nickname, password_hash)
VALUES ('grace@acme.com', 'grace', 'Grace', 'hash_grace')
RETURNING user_id;
```

Result.

```
user_id
-------
7
(1 row)

INSERT 0 1
```

(2) Create a public channel (with authorization check)

This test first lets Alice (`user_id`=1), who is a member of Acme (`workspace_id`=1), create a new public channel `#random`; it then adds her to `channel_members`. The second test lets Eve (`user_id`=5) attempt the same operation in Acme (`workspace_id`=1), but since she is not a member of that workspace, the guarded insert returns no rows.

Query.

```
-- Test A: Alice, who is an Acme member, creates #random.
INSERT INTO channels (workspace_id, name, type, created_by)
SELECT 1, '#random', 'public', 1
WHERE EXISTS (
    SELECT 1 FROM workspace_members
    WHERE workspace_id = 1 AND user_id = 1
)
RETURNING channel_id;

INSERT INTO channel_members (channel_id, user_id)
VALUES (6, 1);

-- Test B: Eve tries to create a channel in Acme, but she is not a
   member.
INSERT INTO channels (workspace_id, name, type, created_by)
SELECT 1, '#eve-unauthorized', 'public', 5
WHERE EXISTS (
    SELECT 1 FROM workspace_members
    WHERE workspace_id = 1 AND user_id = 5
)
RETURNING channel_id;
```

Result.

Test A: Alice, who is an Acme member, creates `#random`.

```
channel_id
----------
6
(1 row)

INSERT 0 1
INSERT 0 1
```

Test B: Eve tries to create a channel in Acme, but she is not a member.

```
channel_id
----------
(0 rows)

INSERT 0 0
```

(3) For each workspace, list all administrators

This query joins `workspaces`, `workspace_members`, and `users` to list every administrator in each workspace.

Query.

```
SELECT w.workspace_id, w.name AS workspace, u.username AS administrator
FROM   workspaces w
JOIN   workspace_members wm ON wm.workspace_id = w.workspace_id
JOIN   users u              ON u.user_id       = wm.user_id
WHERE  wm.is_admin = TRUE
ORDER BY w.workspace_id, u.username;
```

Result.

```
workspace_id | workspace  | administrator
-------------|------------|--------------
1            | Acme Co.   | alice
1            | Acme Co.   | bob
2            | Coop Board | eve
(3 rows)
```

(4) Per public channel: # invitees invited >5 days ago, not yet joined

This query counts, for each public channel in Acme (`workspace_id`=1), the pending invitations that are older than five days and whose invitees still do not appear in `channel_members`.

Query.

```
SELECT c.channel_id, c.name,
       COUNT(ci.invitee_user_id) FILTER (WHERE cm.user_id IS NULL)
       AS stale_unjoined_invites
FROM   channels c
LEFT   JOIN channel_invitations ci
       ON ci.channel_id = c.channel_id
      AND ci.invited_at < CURRENT_TIMESTAMP - INTERVAL '5 days'
LEFT   JOIN channel_members cm
       ON cm.channel_id = ci.channel_id AND cm.user_id =
          ci.invitee_user_id
WHERE  c.workspace_id = 1
  AND  c.type = 'public'
GROUP BY c.channel_id, c.name
ORDER BY c.name;
```

Result.

```
channel_id |   name   | stale_unjoined_invites
-----------+----------+-----------------------
1          | #general |                      1
6          | #random  |                      0
(2 rows)
```

(5) All messages in a channel, chronological order

This test retrieves all messages in Acme `#general` (`channel_id`=1) and orders them by `posted_at` and then by `message_id`.

Query.

```
SELECT m.message_id, m.posted_at, u.username, m.body
FROM   messages m
JOIN   users u ON u.user_id = m.sender_user_id
WHERE  m.channel_id = 1
ORDER BY m.posted_at ASC, m.message_id ASC;
```

Result.

```
message_id |         posted_at          | username |                          body
-----------+-----------------------------+----------+-------------------------------------------------------------
1          | 2026-04-14 21:26:56.771839 | alice    | Welcome to Acme #general!
2          | 2026-04-16 21:26:56.771839 | bob      | Standup notes will be posted here daily.
3          | 2026-04-18 21:26:56.771839 | carol    | Quick geometry question: are these lines perpendicular?
4          | 2026-04-19 21:26:56.771839 | alice    | Let's meet tomorrow at 10am.
(4 rows)
```

(6) All messages posted by a given user

This query lists all messages posted by Alice (`user_id`=1) across all channels and workspaces, ordered from newest to oldest.

Query.

```
SELECT m.message_id, w.name AS workspace, c.name AS channel,
       m.posted_at, m.body
FROM   messages m
JOIN   channels   c ON c.channel_id   = m.channel_id
JOIN   workspaces w ON w.workspace_id = c.workspace_id
WHERE  m.sender_user_id = 1
ORDER BY m.posted_at DESC;
```

Result.

```
message_id | workspace  |    channel     |        posted_at         |                  body
-----------+------------+----------------+--------------------------+-------------------------------------------
4          | Acme Co.   | #general       | 2026-04-19 21:26:56.771839 | Let's meet tomorrow at 10am.
11         | Coop Board | #announcements | 2026-04-17 21:26:56.771839 | Could someone share last month's minutes?
5          | Acme Co.   | #hiring        | 2026-04-15 21:26:56.771839 | Candidate feedback for the backend role.
1          | Acme Co.   | #general       | 2026-04-14 21:26:56.771839 | Welcome to Acme #general!
7          | Acme Co.   | dm-alice-carol | 2026-04-11 21:26:56.771839 | Hey Carol, got a minute?
(5 rows)
```

(7) Accessible messages containing "perpendicular"

This test checks accessibility for three different users: Alice (`user_id`=1), Carol (`user_id`=3), and Dave (`user_id`=4). Alice is a member of both Acme channels that contain the keyword, Carol is only a member of `#general`, and Dave is an Acme workspace member but not a member of any Acme channel.

Query.

```
-- Test A: Alice (user_id = 1)
SELECT m.message_id, w.name AS workspace, c.name AS channel,
       u.username AS author, m.posted_at, m.body
FROM   messages m
JOIN   channels          c  ON c.channel_id    = m.channel_id
JOIN   workspaces        w  ON w.workspace_id  = c.workspace_id
JOIN   users             u  ON u.user_id       = m.sender_user_id
JOIN   channel_members   cm ON cm.channel_id   = c.channel_id
                           AND cm.user_id      = 1
JOIN   workspace_members wm ON wm.workspace_id = w.workspace_id
                           AND wm.user_id      = 1
WHERE  m.body ILIKE '%perpendicular%'
ORDER BY m.posted_at DESC;

-- Test B: Carol (user_id = 3)
SELECT m.message_id, w.name AS workspace, c.name AS channel,
       u.username AS author, m.posted_at, m.body
FROM   messages m
JOIN   channels          c  ON c.channel_id    = m.channel_id
JOIN   workspaces        w  ON w.workspace_id  = c.workspace_id
JOIN   users             u  ON u.user_id       = m.sender_user_id
JOIN   channel_members   cm ON cm.channel_id   = c.channel_id
                           AND cm.user_id      = 3
JOIN   workspace_members wm ON wm.workspace_id = w.workspace_id
                           AND wm.user_id      = 3
WHERE  m.body ILIKE '%perpendicular%'
ORDER BY m.posted_at DESC;

-- Test C: Dave (user_id = 4)
SELECT m.message_id, w.name AS workspace, c.name AS channel,
       u.username AS author, m.posted_at, m.body
```

```
FROM   messages m
JOIN   channels          c  ON c.channel_id    = m.channel_id
JOIN   workspaces        w  ON w.workspace_id  = c.workspace_id
JOIN   users             u  ON u.user_id       = m.sender_user_id
JOIN   channel_members   cm ON cm.channel_id   = c.channel_id
                           AND cm.user_id      = 4
JOIN   workspace_members wm ON wm.workspace_id = w.workspace_id
                           AND wm.user_id      = 4
WHERE  m.body ILIKE '%perpendicular%'
ORDER BY m.posted_at DESC;
```

Result.

Test A: Alice (user_id = 1).

```
message_id | workspace | channel  | author |         posted_at          |                              body
-----------+-----------+----------+--------+----------------------------+-------------------------------------------------------------------
3          | Acme Co.  | #general | carol  | 2026-04-18 21:26:56.771839 | Quick geometry question: are these lines perpendicular?
6          | Acme Co.  | #hiring  | bob    | 2026-04-17 21:26:56.771839 | Their system-design answer had two perpendicular approaches.
(2 rows)
```

Test B: Carol (user_id = 3).

```
message_id | workspace | channel  | author |         posted_at          |                        body
-----------+-----------+----------+--------+----------------------------+-----------------------------------------------------
3          | Acme Co.  | #general | carol  | 2026-04-18 21:26:56.771839 | Quick geometry question: are these lines perpendicular?
(1 row)
```

Test C: Dave (user_id = 4).

```
message_id | workspace | channel | author | posted_at | body
-----------+-----------+---------+--------+-----------+------
(0 rows)
```

(e) System Architecture

snickr follows a classic three-tier web application architecture: a browser front-end, a Flask application server, and a PostgreSQL database. The application logic lives in a single Python module (`app.py`) which exposes routes, executes parameterized SQL via `psycopg2`, and renders results through Jinja2 templates.

Request lifecycle. A typical request flows through the following steps:

1. The browser issues an HTTP request (GET for navigation, POST for form submissions).
2. Flask matches the URL to a route handler in `app.py`.
3. The handler validates the session (`session['user_id']`) and rejects unauthenticated requests.
4. It executes one or more parameterized SQL statements through `psycopg2` (`%s` placeholders, never string concatenation).
5. Multi-step writes are wrapped in `try / commit / rollback` blocks so that either all changes persist or none do.

6. Results are passed to a Jinja2 template, which auto-escapes all dynamic output as a defense against XSS.
7. The rendered HTML is returned to the browser.

(f) Technology Stack

Layer	Technology	Rationale
Backend framework	Python 3 + Flask	Lightweight routing and session handling; minimal setup for an academic demo.
Database driver	`psycopg2`	Native PostgreSQL driver with parameterized queries through `%s` placeholders.
Templating	Jinja2	Built into Flask; HTML auto-escaping is on by default, which is our primary XSS defense.
Database	PostgreSQL 16	Hosts the eight-table relational schema defined in section (b).

(g) Implemented Features

The web layer implements the following features, grouped by category:

- **Authentication**: register, login, logout, edit nickname.
- **Workspace management**: create workspace; invite users (admin-only); promote member to admin; remove member; delete workspace.
- **Channels**: three types (public, private, direct) with type-specific join semantics. Public channels are self-joined from the sidebar, private channels are joined only after a creator-issued invitation, and direct channels are created simultaneously with their two participants.
- **Messages**: post messages, read in chronological order (query 5 from section (c)).
- **Search**: cross-workspace search with permission filtering (query 7 from section (c)).
- **Invitations**: respond to workspace and channel invitations from the profile page.

(h) URL Design and Session State

Session. The application uses Flask's built-in signed-cookie session to store the user's `user_id`. Every protected route begins with the same guard, `if 'user_id' not in session: redirect(login)`, so unauthenticated requests cannot reach handlers that touch user-scoped data.

Bookmarkable URLs. The choice of selected workspace, selected channel, and search keyword is encoded entirely in the URL rather than in the session. The following URLs are all valid bookmarks:

- `/dashboard?ws_id=N&ch_id=M` — open workspace `N`, focused on channel `M`.
- `/workspace_info/<ws_id>` — the member / admin management page for a specific workspace.
- `/search?keyword=xxx` — the search-results page for a given keyword (directly addresses Project 2's requirement to "bookmark the result page of a search").

(i) SQL Injection Defense

All SQL is executed through `psycopg2` using `%s` placeholders, so user-supplied values are bound as parameters by the driver and never interpreted as SQL syntax. There is no string concatenation of user input into SQL anywhere in the codebase.

A representative example from `/register`:

```
cur.execute(
    "INSERT INTO users (email, username, nickname, password_hash) "
    "VALUES (%s, %s, %s, %s)",
    (email, username, nickname, hashed_password)
)
```

The keyword search in `/search` also keeps the user-supplied keyword parameterised; the SQL wildcards are added on the Python side so that the keyword itself is still bound by `psycopg2`:

```
like_keyword = f"%{keyword}%"   # wildcards added in Python
cur.execute(search_query, (user_id, user_id, like_keyword))
```

(j) XSS Defense

Cross-site scripting is prevented by relying on Jinja2's default behaviour: every `{{ ... }}` expression in a template is HTML-escaped before being written to the response.

For example, the message body in the chat view is rendered as `<div class="msg-body">{{ m.body }}</div>`. If a user submits the text `<script>alert(1)</script>`, the database stores it verbatim but the template emits `&lt;script&gt;alert(1)&lt;/script&gt;`, which the browser displays as literal text rather than executing.

(k) Transactions and Concurrency

Multi-step database operations are wrapped in `try / commit / rollback` blocks so that either all rows are written or none of them are. The list below covers every route where a transaction is required.

`/create_workspace`
INSERT workspace + INSERT admin member. *Rollback* on `UNIQUE` violation.

`/create_channel`
INSERT channel + INSERT channel member(s) — one for public/private (the creator), two for direct (both participants). *Rollback* on duplicate name or any failed step.

`/respond_invite` (accept)
UPDATE `workspace_invitations.status` + INSERT into `workspace_members`. *Rollback* on conflict; declined invitations skip the second step.

`/respond_channel_invite` (accept)
UPDATE `channel_invitations.status` + INSERT into `channel_members`. *Rollback* on conflict; declined invitations skip the second step.

`/delete_workspace`
DELETE from `messages` → `channel_members` → `channel_invitations` → `channels` → `workspace_invitations` → `workspace_members` → `workspaces`. *Rollback* preserves the entire workspace if any DELETE fails.

`/remove_member`
: DELETE the user's rows in `channel_members`, `channel_invitations`, `workspace_invitations`, and `workspace_members` (scoped to the workspace). *Rollback* preserves the user's complete state.

Database-level concurrency guards. The schema constraints listed below prevent inconsistent state when two requests race against each other. If a second transaction tries to insert a conflicting row, it receives an `IntegrityError` which the application catches and reports to the user.

- `UNIQUE(email)` and `UNIQUE(username)` on `users`: prevent duplicate account creation.
- `UNIQUE(workspace_id, name)` on `channels`: prevents duplicate channel names in the same workspace (and, indirectly, duplicate direct-message channels through the `dm-{u1}-{u2}` naming convention).
- `UNIQUE(created_by, name)` on `workspaces`: prevents the same user from creating two workspaces with the same name.
- `PRIMARY KEY (workspace_id, user_id)` on `workspace_members`: prevents duplicate workspace memberships.
- `PRIMARY KEY (channel_id, user_id)` on `channel_members`: prevents duplicate channel memberships.
- `PRIMARY KEY (workspace_id, invitee_user_id)` on `workspace_invitations`: enforces one invitation per (workspace, invitee) pair.
- `PRIMARY KEY (channel_id, invitee_user_id)` on `channel_invitations`: enforces one invitation per (channel, invitee) pair.

(l) User Guide

A typical user journey through the system is summarised below.

1. **Sign up**: visit `/register`, enter an email, a unique username, an optional nickname, and a password.
2. **Log in**: `/login` with username and password. On success the user lands on the dashboard.
3. **Edit profile**: click the username in the dashboard sidebar to open `/profile`; the Edit Profile block at the top updates the nickname.
4. **Create a workspace**: use the "+ New Workspace" form in the sidebar. The creator automatically becomes the workspace's initial admin.
5. **Invite users**: from the workspace info page, admins can use the "Invite a User" form to send a pending invitation.
6. **Respond to invitations**: `/profile` surfaces both workspace and channel invitations with Accept/Decline buttons.
7. **Manage members**: the workspace info page shows separate Administrators and Members tables. Admins see "Make Admin" and "Remove" buttons in the Members table.
8. **Create a channel**: use the "+ Create Channel" form in the sidebar. Select *public* (anyone in the workspace can self-join), *private* (only the creator can invite), or *direct* (a recipient is chosen from a dropdown of workspace members and both users are added).

9. **Join a public channel**: public channels the user has not yet joined appear dimmed in the sidebar with a green "Join" button.
10. **Invite to a private channel**: when viewing a private channel the user created, an Invite form appears next to the search box; the recipient is selected from the workspace-member dropdown.
11. **Send messages**: type into the input box at the bottom of the channel view and submit. Messages appear in chronological order.
12. **Search**: use the search box in the top-right of the channel view; results are filtered so the user only sees messages from channels and workspaces they belong to.
13. **Log out**: click "Logout" in the sidebar to clear the session.

Appendix A: GitHub Repository

The complete source code for snickr is hosted on the `dev` branch at
`https://github.com/Zihangyu600/snickr/tree/dev`
The repository contains:

- `app.py` — all Flask routes and database logic.
- `templates/` — the seven HTML templates (login, register, dashboard, profile, workspace_info, search, and the shared `base.html`).
- `sql/` — `schema.sql` and `sample_data.sql` bundled for self-contained setup.
- `README.md` — end-to-end setup instructions (create database, load schema, configure connection, install dependencies, run the app).

Appendix B: Route Reference

The application exposes the following routes. The *Auth* column uses - for public routes, ✓ for routes requiring an authenticated session, `admin` for routes requiring workspace administrator privileges, and `creator` for routes restricted to the creator of the affected workspace or channel.

Authentication

Route	Method	Auth	Purpose
`/register`	GET/POST	–	Create a user account
`/login`	GET/POST	–	Sign in
`/logout`	GET	✓	Clear the session

Dashboard and messages

Route	Method	Auth	Purpose
`/dashboard`	GET	✓	Main chat view (workspace/channel list, messages)
`/send_message`	POST	✓	Post a message to the current channel
`/search`	GET	✓	Keyword search across accessible content

Workspaces

Route	Method	Auth	Purpose
`/create_workspace`	POST	✓	Create a workspace (transaction)
`/workspace_info/<ws_id>`	GET	✓	Workspace details, members, invitations
`/invite_user`	POST	admin	Invite a user to the workspace
`/respond_invite`	POST	✓	Accept / decline a workspace invitation
`/promote_admin`	POST	admin	Promote a member to admin
`/remove_member`	POST	admin	Remove a member (transaction)
`/delete_workspace/<ws_id>`	POST	creator	Delete the workspace (transaction)

Channels

Route	Method	Auth	Purpose
`/create_channel`	POST	✓	Create a channel; the direct branch creates a DM
`/join_channel`	POST	✓	Self-join a public channel
`/invite_to_channel`	POST	creator	Invite a user to a private channel
`/respond_channel_invite`	POST	✓	Accept / decline a channel invitation

Profile

Route	Method	Auth	Purpose
`/profile`	GET	✓	Pending invitations, message history, profile editor
`/update_nickname`	POST	✓	Update the user's nickname

Appendix C: User Interface

The figures below show the main views of the deployed system, captured while following the user guide in section (l).

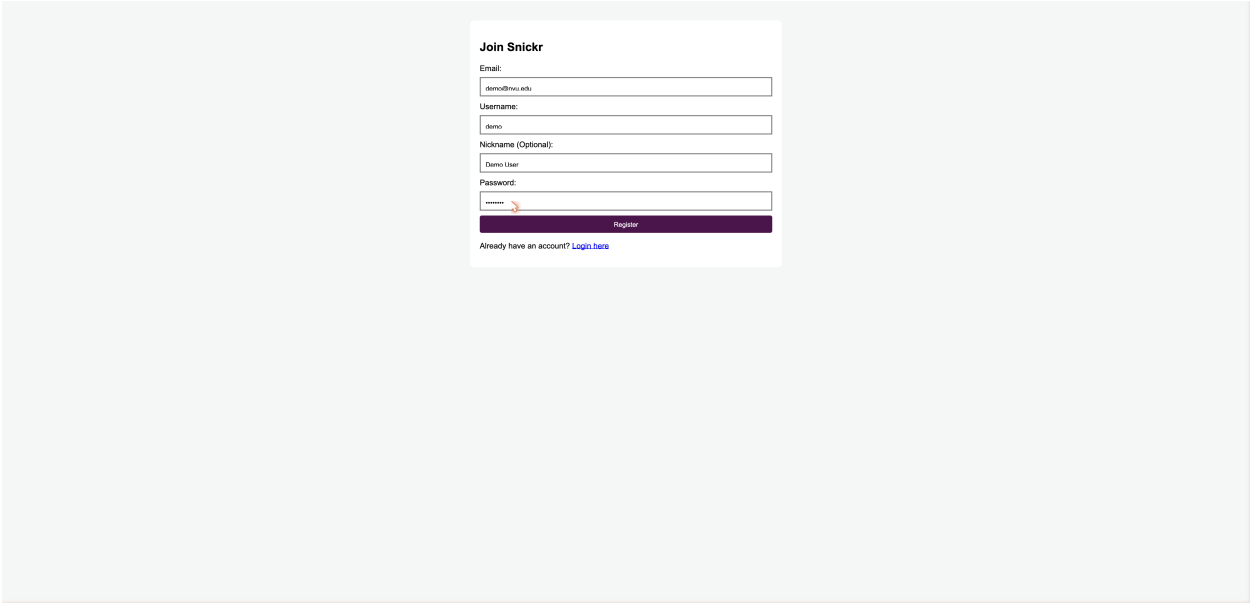


Figure 1: Registration page. The user supplies an email, username, optional nickname, and password.

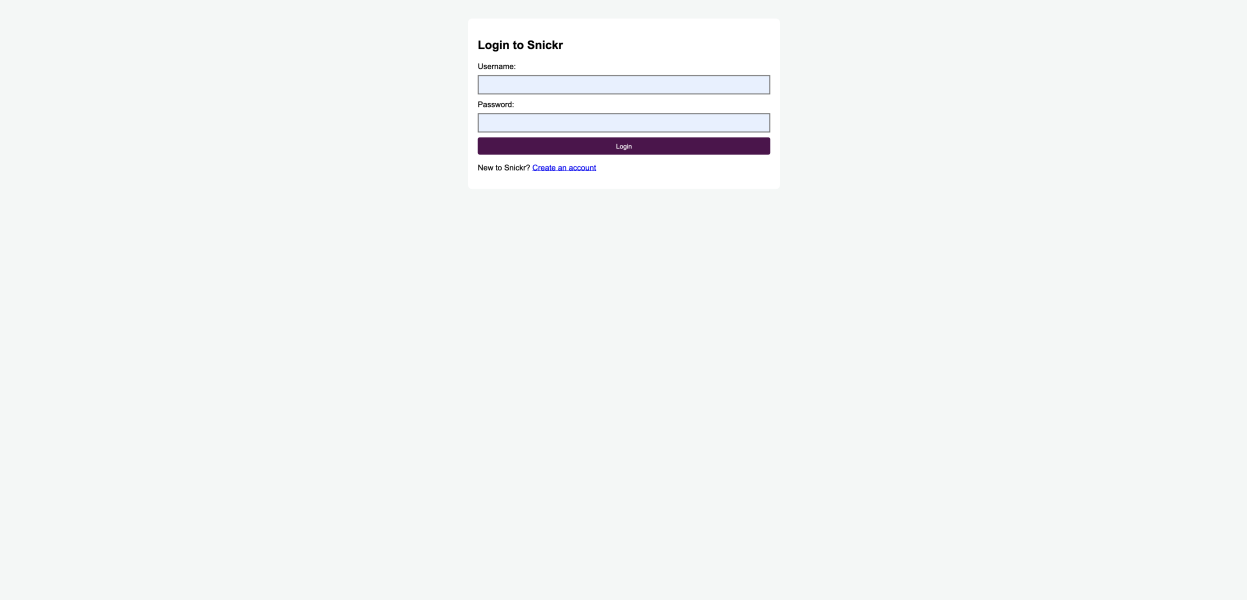


Figure 2: Login page. After successful authentication the user is redirected to the dashboard.

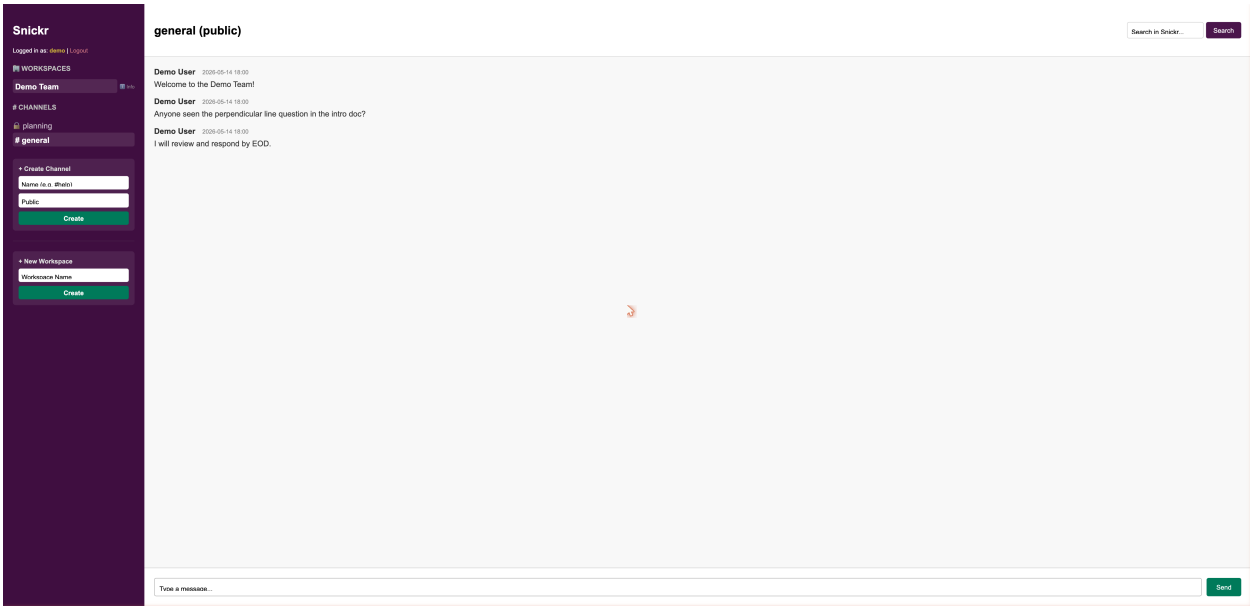


Figure 3: Main dashboard. The sidebar lists the active workspace, the joined channels (private channels prefixed with a lock icon), and forms to create channels and workspaces. The right panel shows the chronological message list for the selected channel, a search box, and the message input.

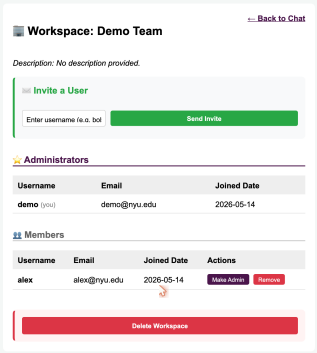


Figure 4: Workspace info page. Administrators are listed in the top table; ordinary members are listed below with per-row *Make Admin* and *Remove* actions. The Invite a User form is admin-only, and the *Delete Workspace* button only appears for the workspace creator.

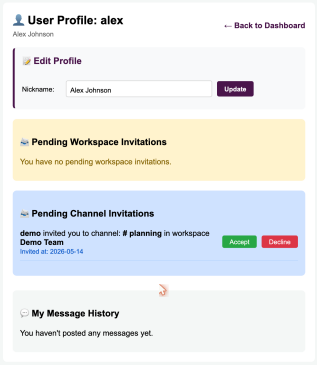


Figure 5: Profile page. The top block lets the user update their nickname. Pending workspace and channel invitations are surfaced in separate blocks; each invitation offers Accept and Decline buttons. The user's message history is listed at the bottom.

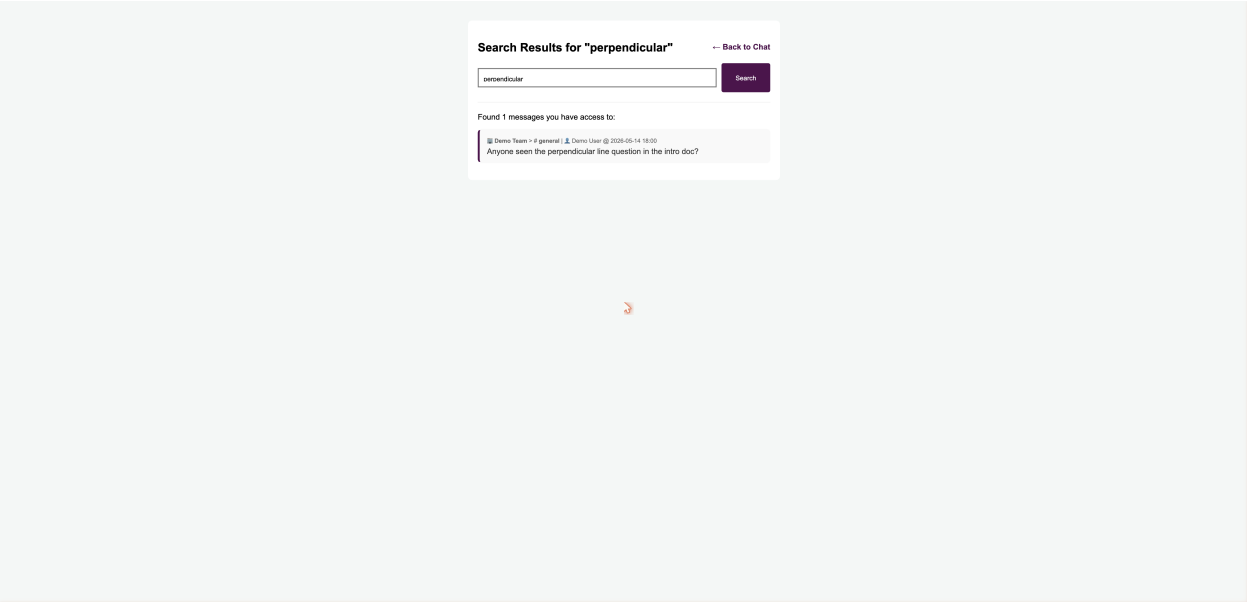


Figure 6: Search results. The query is preserved in the URL (`/search?keyword=...`) so the page is bookmarkable. Results are filtered to only include messages from channels and workspaces the current user belongs to.